```
import React, { useEffect, useMemo, useState } from "react";
import { createRoot } from "react-dom/client";
import {
  Activity, ArrowDownRight, ArrowRight, BookOpen, Bookmark, CalendarDays,
  Check, ChevronDown, CircleHelp, Clock3, Compass, GraduationCap, LayoutDashboard,
  LogOut, Menu, MessageCircle, Plus, Search, Send, Settings, Shield, Sparkles,
  Target, Timer, Trophy, Users, X, Zap
} from "lucide-react";
import { supabase, configured } from "./supabase";
import WriteArticle from "./WriteArticle";
import Messages from "./Messages";
import SocialHub from "./SocialHub";
import "./style.css";
import "./unblinded-redesign.css";

const navItems = [
  ["home", "Overview", LayoutDashboard],
  ["learn", "Learn", BookOpen],
  ["community", "Community", Users],
  ["social", "Social feed", MessageCircle],
  ["messages", "Messages", MessageCircle],
  ["planner", "Planner", CalendarDays],
  ["tools", "Study tools", Zap],
  ["opportunities", "Opportunities", Trophy]
];
const levels = ["Class 6–8", "SSC", "HSC", "Admission", "University", "Professional"];
const subjects = ["Mathematics", "Physics", "Chemistry", "Biology", "English", "Bangla",
"ICT", "Accounting", "Economics", "Other"];
const fmtDate = value => value ? new Date(value).toLocaleDateString("en-GB", { day:
"numeric", month: "short" }) : "—";

function App() {
  const [session, setSession] = useState(null);
  const [profile, setProfile] = useState(null);
  const [page, setPage] = useState("home");
  const [mobileNav, setMobileNav] = useState(false);
  const [dark, setDark] = useState(false);
  const [notice, setNotice] = useState("");
  const [loading, setLoading] = useState(true);
  const [articles, setArticles] = useState([]);
  const [tasks, setTasks] = useState([]);
  const [opportunities, setOpportunities] = useState([]);
  const [chat, setChat] = useState([]);
  const [query, setQuery] = useState("");
  const [levelFilter, setLevelFilter] = useState("All levels");
  const [subjectFilter, setSubjectFilter] = useState("All subjects");
  const [authMode, setAuthMode] = useState("login");
  const [auth, setAuth] = useState({ name: "", email: "", password: "" });
  const [busy, setBusy] = useState(false);
  const [taskTitle, setTaskTitle] = useState("");
  const [taskDue, setTaskDue] = useState("");
  const [taskKind, setTaskKind] = useState("Assignment");
  const [chatText, setChatText] = useState("");
  const [selectedArticle, setSelectedArticle] = useState(null);
  const [comments, setComments] = useState([]);
  const [commentText, setCommentText] = useState("");
  const [gpa, setGpa] = useState({ obtained: "", total: "5" });
  const [timerMinutes, setTimerMinutes] = useState(25);
  const [secondsLeft, setSecondsLeft] = useState(25 * 60);
  const [timerRunning, setTimerRunning] = useState(false);
  const [admin, setAdmin] = useState(false);
  const [pending, setPending] = useState([]);

  const say = message => { setNotice(message); window.setTimeout(() => setNotice(""), 3200);
```

```
};

  useEffect(() => {
    if (!configured) { setLoading(false); return; }
    supabase.auth.getSession().then(({ data }) => { setSession(data.session);
setLoading(false); });
    const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, value) =>
setSession(value));
    return () => subscription.unsubscribe();
  }, []);

  useEffect(() => {
    if (!session || !configured) { setProfile(null); setAdmin(false); return; }
    (async () => {
      const { data } = await supabase.from("profiles").select("*").eq("id",
session.user.id).maybeSingle();
      setProfile(data);
      setAdmin(["admin", "moderator"].includes(data?.role));
      await supabase.from("student_profiles").upsert({ user_id: session.user.id }, {
onConflict: "user_id", ignoreDuplicates: true });
    })();
  }, [session]);

  useEffect(() => {
    if (!configured) return;
    loadArticles();
    loadTasks();
    loadOpportunities();
    loadChat();
  }, [session, page]);

  useEffect(() => {
    if (page === "admin" && admin) loadPending();
  }, [page, admin]);

  useEffect(() => {
    if (!timerRunning) return;
    const id = window.setInterval(() => setSecondsLeft(s => {
      if (s <= 1) { window.clearInterval(id); setTimerRunning(false); say("Study session
complete. Take a short break."); return 0; }
      return s - 1;
    }), 1000);
    return () => window.clearInterval(id);
  }, [timerRunning]);

  async function loadArticles() {
    if (!configured) return;
    const { data, error } = await supabase.from("articles")

.select("id,title,slug,excerpt,body,category,created_at,author_id,profiles(display_name,avat
ar_url)")
      .eq("status", "published").order("created_at", { ascending: false }).limit(80);
    if (!error) setArticles(data || []);
  }
  async function loadTasks() {
    if (!session || !configured) { setTasks([]); return; }
    const { data } = await supabase.from("study_tasks").select("*")
      .eq("user_id", session.user.id).order("due_date", { ascending: true });
    setTasks(data || []);
  }
  async function loadOpportunities() {
    if (!configured) return;
    const { data } = await supabase.from("opportunities").select("*")
      .eq("status", "published").order("deadline", { ascending: true }).limit(30);
    setOpportunities(data || []);
```

```
  }
  async function loadChat() {
    if (!configured) return;
    const { data } = await supabase.from("chat_messages")
      .select("id,body,created_at,profiles(display_name)")
      .eq("status", "visible").order("created_at", { ascending: true }).limit(80);
    setChat(data || []);
  }
  async function loadPending() {
    const { data } = await
supabase.from("articles").select("id,title,category,created_at,profiles(display_name)")
      .eq("status", "pending").order("created_at");
    setPending(data || []);
  }
  async function signIn(event) {
    event.preventDefault(); setBusy(true);
    let result;
    if (authMode === "signup") {
      result = await supabase.auth.signUp({ email: auth.email, password: auth.password,
options: { data: { display_name: auth.name } } });
      if (!result.error && result.data.user) {
        await supabase.from("profiles").upsert({ id: result.data.user.id, display_name:
auth.name });
      }
    } else result = await supabase.auth.signInWithPassword({ email: auth.email, password:
auth.password });
    setBusy(false);
    if (result.error) say(result.error.message);
    else { setPage("home"); say(authMode === "signup" ? "Account created. Check your email
if confirmation is enabled." : "Welcome back."); }
  }
  async function addTask(event) {
    event.preventDefault();
    if (!session) return say("Sign in to add tasks.");
    if (!taskTitle.trim()) return;
    const { error } = await supabase.from("study_tasks").insert({
      user_id: session.user.id, title: taskTitle.trim(), task_type: taskKind,
      due_date: taskDue || null, status: "todo"
    });
    if (error) say(error.message);
    else { setTaskTitle(""); setTaskDue(""); await loadTasks(); say("Task added."); }
  }
  async function toggleTask(task) {
    const { error } = await supabase.from("study_tasks").update({
      status: task.status === "done" ? "todo" : "done"
    }).eq("id", task.id);
    if (error) say(error.message); else loadTasks();
  }
  async function removeTask(task) {
    const { error } = await supabase.from("study_tasks").delete().eq("id", task.id);
    if (error) say(error.message); else loadTasks();
  }
  async function sendChat(event) {
    event.preventDefault();
    if (!session) return say("Sign in to join the student discussion.");
    if (!chatText.trim()) return;
    const { error } = await supabase.from("chat_messages").insert({ author_id:
session.user.id, body: chatText.trim() });
    if (error) say(error.message); else { setChatText(""); loadChat(); }
  }
  async function openArticle(article) {
    setSelectedArticle(article); setPage("resource");
    const { data } = await
supabase.from("comments").select("id,body,created_at,profiles(display_name)")
      .eq("article_id", article.id).eq("status", "visible").order("created_at");
```

```
    setComments(data || []);
  }
  async function sendComment(event) {
    event.preventDefault();
    if (!session) return say("Sign in to ask or comment.");
    if (!commentText.trim() || !selectedArticle) return;
    const { error } = await supabase.from("comments").insert({
      article_id: selectedArticle.id, author_id: session.user.id, body: commentText.trim()
    });
    if (error) say(error.message);
    else { setCommentText(""); const { data } = await supabase.from("comments")
      .select("id,body,created_at,profiles(display_name)").eq("article_id",
selectedArticle.id)
      .eq("status", "visible").order("created_at"); setComments(data || []); }
  }
  async function moderateArticle(id, status) {
    const { error } = await supabase.from("articles").update({ status }).eq("id", id);
    say(error?.message || "Review updated."); loadPending();
  }
  async function updateStudent(field, value) {
    if (!session) return;
    const { error } = await supabase.from("student_profiles").upsert({
      user_id: session.user.id, [field]: value, updated_at: new Date().toISOString()
    }, { onConflict: "user_id" });
    if (error) say(error.message); else say("Profile saved.");
  }

  const filteredArticles = useMemo(() => articles.filter(a => {
    const text = `${a.title || ""} ${a.excerpt || ""} ${a.category || ""}`.toLowerCase();
    const q = query.toLowerCase().trim();
    const categoryMatch = subjectFilter === "All subjects" || (a.category ||
"").toLowerCase().includes(subjectFilter.toLowerCase());
    return (!q || text.includes(q)) && categoryMatch;
  }), [articles, query, subjectFilter]);
  const doneCount = tasks.filter(t => t.status === "done").length;
  const progress = tasks.length ? Math.round(doneCount / tasks.length * 100) : 0;
  const firstName = profile?.display_name?.split(" ")[0] || "there";

  if (!configured) return <div className="config-screen"><div
className="brand">UNBLINDED<span>.</span></div><h1>Your learning space starts here.</h1>
<p>Add the existing VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY environment variables to
your hosting project.</p></div>;
  if (loading) return <div className="loading-screen"><div
className="brand">UNBLINDED<span>.</span></div><div className="loader"/></div>;

  const Sidebar = () => <aside className={`sidebar ${mobileNav ? "sidebar-open" : ""}`}>
    <a className="brand" onClick={() => setPage("home")}>UNBLINDED<span>.</span></a>
    <div className="workspace-label">STUDENT WORKSPACE</div>
    <nav>{navItems.map(([id, label, Icon]) => <button key={id} className={page === id ?
"nav-item active" : "nav-item"} onClick={() => { setPage(id); setMobileNav(false); }}><Icon
size={17}/><span>{label}</span>{id === "messages" && <i className="nav-dot"/>}</button>)}
</nav>
    <div className="sidebar-bottom">
      {admin && <button className="nav-item" onClick={() =>
{setPage("admin");setMobileNav(false);}}><Shield size={17}/> Administration</button>}
      <div className="sidebar-user"><div className="avatar">{(profile?.display_name ||
"S").slice(0,1).toUpperCase()}</div><div><b>{profile?.display_name || "Student"}</b><small>
{session.user.email}</small></div><button title="Sign out" onClick={() =>
supabase.auth.signOut()}><LogOut size={16}/></button></div>
    </div>
  </aside>;

  const Header = ({ title, subtitle }) => <header className="page-header">
    <button className="mobile-menu" onClick={() => setMobileNav(!mobileNav)}><Menu size=
{20}/></button>
```

```
    <div><div className="eyebrow">UNBLINDED / {title.toUpperCase()}</div><h1>{title}</h1>
{subtitle && <p>{subtitle}</p>}</div>
    <div className="header-actions"><span className="region-pill">Bangladesh edition</span>
<button className="icon-button" title="Profile" onClick={() => setPage("profile")}><Settings
size={17}/></button></div>
  </header>;

  const TaskList = ({ limit }) => <div className="task-list">{tasks.slice(0, limit ||
20).map(task => <div className={`task-row ${task.status === "done" ? "task-done" : ""}`}
key={task.id}><button className="task-check" onClick={() => toggleTask(task)}>{task.status
=== "done" ? <Check size={14}/> : null}</button><div className="task-copy"><b>{task.title}
</b><small>{task.task_type} · {fmtDate(task.due_date)}</small></div><button className="task-
remove" onClick={() => removeTask(task)}><X size={14}/></button></div>)}{!tasks.length &&
<div className="empty-state">No tasks yet. Add your first study goal.</div>}</div>;

  return <div className={dark ? "app-shell dark" : "app-shell"}>
    <Sidebar/>
    {mobileNav && <button className="mobile-scrim" onClick={() => setMobileNav(false)} aria-
label="Close navigation"/>}
    <main className="main-area">
      {page === "home" && <><Header title="Overview" subtitle={`A clearer view of your
learning journey, ${firstName}.`}/>
        <section className="welcome-panel"><div><div className="welcome-kicker"><Sparkles
size={14}/> YOUR SPACE TO GROW</div><h2>Make today<br/><em>count.</em></h2><p>Learn with
intention. Build good habits. Find your next opportunity.</p><button className="primary-
button" onClick={() => setPage("learn")}>Explore learning resources <ArrowRight size={16}/>
</button></div><div className="welcome-art"><div className="orbit orbit-one"/><div
className="orbit orbit-two"/><GraduationCap size={74} strokeWidth={1}/><span>LEARN · CONNECT
· GROW</span></div></section>
        <div className="metric-grid"><div className="metric-card"><div className="metric-
icon blue"><Target size={18}/></div><small>Tasks completed</small><strong>{doneCount}<span>
/ {tasks.length}</span></strong><div className="mini-progress"><i style=
{{width:`${progress}%`}}/></div><label>{progress}% of your task list</label></div><div
className="metric-card"><div className="metric-icon violet"><BookOpen size={18}/></div>
<small>Learning resources</small><strong>{articles.length}</strong><label>Available to
explore</label></div><div className="metric-card"><div className="metric-icon teal">
<CalendarDays size={18}/></div><small>Upcoming deadlines</small><strong>{tasks.filter(t =>
t.status !== "done" && t.due_date).length}</strong><label>Keep your momentum</label></div>
<div className="metric-card"><div className="metric-icon amber"><Trophy size={18}/></div>
<small>Opportunities</small><strong>{opportunities.length}</strong><label>Discover what is
next</label></div></div>
        <div className="content-grid"><section className="surface-card"><div
className="card-heading"><div><span className="eyebrow">YOUR PRIORITIES</span><h3>Upcoming
tasks</h3></div><button className="text-action" onClick={() => setPage("planner")}>View
planner <ArrowRight size={14}/></button></div><TaskList limit={5}/></section><section
className="surface-card quick-card"><span className="eyebrow">QUICK ACCESS</span><h3>Make
progress, one step at a time.</h3><div className="quick-links"><button onClick={() =>
setPage("planner")}><CalendarDays/>Plan your week<ArrowRight/></button><button onClick={()
=> setPage("tools")}><Timer/>Start a focus session<ArrowRight/></button><button onClick={()
=> setPage("opportunities")}><Compass/>Explore opportunities<ArrowRight/></button></div>
</section></div>
      </>}
      {page === "social" && <><Header title="Social feed" subtitle="Share ideas, discover
perspectives and connect with the UNBLINDED community."/><SocialHub supabase={supabase}
session={session} profile={profile}/></>}
      {page === "learn" && <><Header title="Learn" subtitle="A growing library of notes,
guides and ideas."/><div className="toolbar-panel"><div className="search-field"><Search
size={17}/><input value={query} onChange={e=>setQuery(e.target.value)} placeholder="Search
resources, topics or keywords..."/></div><select value={subjectFilter} onChange=
{e=>setSubjectFilter(e.target.value)}><option>All subjects</option>{subjects.map(s=><option
key={s}>{s}</option>)}</select></div><div className="filter-chips">{["All
levels",...levels].map(l=><button key={l} className={levelFilter===l?"chip active":"chip"}
onClick={()=>setLevelFilter(l)}>{l}</button>)}</div><div className="resource-grid">
{filteredArticles.map(a=><article className="resource-card" key={a.id} onClick=
{()=>openArticle(a)}><div className="resource-top"><span className="resource-type">
```

```
{a.category || "Learning resource"}</span><Bookmark size={16}/></div><h3>{a.title}</h3><p>
{a.excerpt || (a.body || "").replace(/<[^>]+>/g," ").slice(0,150)}</p><div
className="resource-meta"><span>{a.profiles?.display_name || "UNBLINDED contributor"}</span>
<span>{fmtDate(a.created_at)}</span></div></article>)}</div>{!filteredArticles.length&&<div
className="empty-state large">No published resources found. Try another search.</div>}<div
className="submit-resource"><div><h3>Have something useful to share?</h3><p>Contribute
notes, explainers or study guides to the student library.</p></div><button
className="primary-button" onClick={()=>setPage(session?"write":"auth")}>Contribute a
resource <ArrowRight size={16}/></button></div></>}
      {page === "resource" && selectedArticle && <><Header title="Resource" subtitle="Read,
reflect and discuss."/><article className="reading-card"><button className="text-action"
onClick={()=>setPage("learn")}>← Back to library</button><span className="eyebrow">
{selectedArticle.category} · {fmtDate(selectedArticle.created_at)}</span><h2>
{selectedArticle.title}</h2><p className="reading-author">By
{selectedArticle.profiles?.display_name || "UNBLINDED contributor"}</p><div
className="reading-body" dangerouslySetInnerHTML={{__html:selectedArticle.body || ""}}/><div
className="discussion-block"><h3>Questions & discussion</h3>{comments.map(c=><div
className="comment-row" key={c.id}><b>{c.profiles?.display_name || "Student"}</b><p>{c.body}
</p><small>{fmtDate(c.created_at)}</small></div>)}<form onSubmit={sendComment}
className="inline-form"><input value={commentText} onChange=
{e=>setCommentText(e.target.value)} placeholder="Ask a question or add a thought..."/>
<button className="primary-button"><Send size={15}/> Post</button></form></div></article>
</>}
      {page === "community" && <><Header title="Community" subtitle="Find your people. Share
what you know. Build together."/>
        <section className="discord-workspace">
          <aside className="discord-channels">
            <div className="discord-server-mark">U<span>.</span></div>
            <div className="discord-server-title"><b>UNBLINDED HUB</b><small>Learn · Create
· Grow</small></div>
            <div className="discord-channel-group">START HERE</div>
            <div className="discord-channel active"><span>#</span> welcome</div>
            <div className="discord-channel"><span>#</span> announcements</div>
            <div className="discord-channel"><span>#</span> introductions</div>
            <div className="discord-channel-group">COMMUNITY</div>
            <div className="discord-channel"><span>#</span> general</div>
            <div className="discord-channel"><span>#</span> share-your-work</div>
            <div className="discord-channel"><span>#</span> questions</div>
            <div className="discord-channel-group">LEARNING</div>
            <div className="discord-channel"><span>#</span> study-room</div>
            <div className="discord-channel"><span>#</span> resources</div>
            <div className="discord-channel-group">VOICE ROOMS</div>
            <div className="discord-channel voice"><span>◖</span> Study room</div>
            <div className="discord-channel voice"><span>◖</span> Open lounge</div>
            <div className="discord-channel voice"><span>◖</span> Gaming room</div>
            <div className="discord-channel-foot">A community built on respect.</div>
          </aside>
          <section className="discord-chat">
            <div className="discord-chat-top"><div><b># general</b><span>Chat, share ideas
and learn together.</span></div><span className="discord-online-pill">Community space</span>
</div>
            <div className="discord-welcome"><div className="discord-welcome-icon"><Sparkles
size={22}/></div><div><span className="eyebrow">WELCOME TO UNBLINDED</span><h2>A place to
think out loud.</h2><p>Meet curious people, exchange knowledge and make something meaningful
together.</p></div></div>
            <div className="discord-prompts"><div><b>✦ Be curious</b><small>Ask thoughtful
questions</small></div><div><b>↗ Share knowledge</b><small>Help someone move forward</small>
</div><div><b>♡ Be kind</b><small>Make room for everyone</small></div></div>
            <div className="discord-feed-label">RECENT CONVERSATION</div>
            <div className="discord-message-list">{chat.map(m=><article className="discord-
message" key={m.id}><div className="discord-message-avatar">
{(m.profiles?.display_name||"U").slice(0,1).toUpperCase()}</div><div><div
className="discord-message-meta"><b>{m.profiles?.display_name||"UNBLINDED member"}</b>
<small>{fmtDate(m.created_at)}</small></div><p>{m.body}</p></div></article>)}{!chat.length&&
```

No messages yet. Start the first conversation below. }

            <form onSubmit={sendChat} className="discord-composer"><input value={chatText}
onChange={e=>setChatText(e.target.value)} maxLength={1000} placeholder={session?"Message
#general":"Sign in to join the conversation"}/><button className="primary-button" disabled=
{!session||!chatText.trim()}><Send size={16}/></button></form>
            <Shield size={14}/> Keep it respectful. Protect your
privacy and the privacy of others.
          </section>
          <aside className="discord-members"> YOUR
SPACE
{(profile?.display_name||"U").slice(0,1).toUpperCase()}
{profile?.display_name||"Your profile"}<small>Signed in member</small> <i/>
COMMUNITY GUIDELINES
Respect different perspectives. Share useful context. Keep personal information private.
MAKE IT BETTER
Introduce yourself, share a project or help someone with a question. </aside>
        </section></>}
      {page === "messages" && <><Header title="Messages" subtitle="Continue your one-to-one
conversations."/><Messages supabase={supabase} onClose={()=>setPage("home")}/></>}
      {page === "planner" && <><Header title="Planner" subtitle="Turn your intentions into a
manageable plan."/> <section className="surface-card"><div
className="card-heading"> TASK MANAGER<h3>Your study
plan</h3> {tasks.length} tasks <form
className="task-form" onSubmit={addTask}><input value={taskTitle} onChange=
{e=>setTaskTitle(e.target.value)} placeholder="What do you need to work on?" required/>
<select value={taskKind} onChange={e=>setTaskKind(e.target.value)}>
<option>Assignment</option><option>Exam preparation</option><option>Reading</option>
<option>Personal goal</option></select><input type="date" value={taskDue} onChange=
{e=>setTaskDue(e.target.value)}/><button className="primary-button"><Plus size={16}/> Add
task</button></form><TaskList/></section><section className="surface-card progress-card">
CONSISTENCY OVER INTENSITY<h3>Your progress</h3><div
className="progress-ring" style={{"--progress":`${progress*3.6}deg`}}>
{progress}%<small>completed</small> {doneCount} of {tasks.length}
tasks completed. Small steps add up. </section> </>}
      {page === "tools" && <><Header title="Study tools" subtitle="Simple utilities to
support better study habits."/> <section className="surface-card
tool-card"> <Timer/> FOCUS
TIMER<h3>Pomodoro session</h3>
{String(Math.floor(secondsLeft/60)).padStart(2,"0")}:
{String(secondsLeft%60).padStart(2,"0")} <button
className="primary-button" onClick={()=>setTimerRunning(!timerRunning)}>
{timerRunning?"Pause":"Start focus"}</button><button className="secondary-button" onClick=
{()=>{setTimerRunning(false);setSecondsLeft(timerMinutes*60);}}>Reset</button> <div
className="timer-presets">{[15,25,45,60].map(n=><button className={timerMinutes===n?"chip
active":"chip"} key={n} onClick={()=>
{setTimerMinutes(n);setSecondsLeft(n*60);setTimerRunning(false);}}>{n} min</button>)}
</section><section className="surface-card tool-card">
<Activity/> GRADE ESTIMATOR<h3>GPA percentage
helper</h3> Estimate a percentage against a maximum score. This is not an official
Bangladesh GPA conversion. <label>Score obtained<input
type="number" min="0" value={gpa.obtained} onChange=
{e=>setGpa({...gpa,obtained:e.target.value})}/></label><label>Maximum score<input
type="number" min="1" value={gpa.total} onChange=
{e=>setGpa({...gpa,total:e.target.value})}/></label>
{gpa.obtained!==""&&Number(gpa.total)>0?
`${Math.max(0,Math.min(100,Number(gpa.obtained)/Number(gpa.total)*100)).toFixed(1)}%`:"—"}
<small>Percentage estimate only · verify your institution's grading rules.</small>
</section><section className="surface-card tool-card">
<Target/> STUDY METHOD<h3>Active recall</h3> After
reading a topic, close your notes and write down everything you remember. Review the gaps,
then repeat tomorrow. <button className="secondary-button" onClick=
{()=>setPage("learn")}>Find a topic <ArrowRight size={15}/></button></section> </>}
      {page === "opportunities" && <><Header title="Opportunities" subtitle="Find ways to
learn, participate and move forward."/> <Sparkles/>

```
<h2>Your next opportunity may be one click away.</h2><p>Explore scholarships, competitions,
internships and learning programmes.</p></div></div><div className="opportunity-grid">
{opportunities.map(o=><article className="opportunity-card" key={o.id}><span
className="resource-type">{o.category||"Opportunity"}</span><h3>{o.title}</h3><p>
{o.description}</p><div className="opportunity-meta"><span>{o.location||"Bangladesh /
Online"}</span><span>Deadline: {fmtDate(o.deadline)}</span></div>{o.application_url&&<a
href={o.application_url} target="_blank" rel="noreferrer" className="text-action">View
opportunity <ArrowDownRight size={15}/></a>}</article>)}</div>{!opportunities.length&&<div
className="empty-state large">New opportunities will appear here after they are verified and
published.</div>}</>}
      {page === "write" && <><Header title="Contribute" subtitle="Share useful knowledge
with the student community."/><WriteArticle supabase={supabase} onClose=
{()=>setPage("learn")}/></>}
      {page === "profile" && <><Header title="Your profile" subtitle="Set up the academic
context that helps shape your workspace."/><StudentProfile session={session} profile=
{profile} updateStudent={updateStudent}/></>}
      {page === "admin" && admin && <><Header title="Administration" subtitle="Review
submitted learning resources."/><section className="surface-card"><div className="card-
heading"><h3>Pending resource submissions</h3><span className="count-pill">{pending.length}
pending</span></div>{pending.map(a=><div className="admin-row" key={a.id}><div><b>{a.title}
</b><small>{a.category} · {a.profiles?.display_name||"Student"}</small></div><div><button
className="primary-button" onClick={()=>moderateArticle(a.id,"published")}>Approve</button>
<button className="secondary-button" onClick=
{()=>moderateArticle(a.id,"rejected")}>Decline</button></div></div>)}{!pending.length&&<div
className="empty-state">Nothing waiting for review.</div>}</section></>}
      <footer className="app-footer"><div className="brand">UNBLINDED<span>.</span></div>
<span>Your space to learn, connect and grow.</span><span>© 2026 UNBLINDED</span></footer>
    </main>
    {notice&&<div className="toast">{notice}</div>}
    {page==="auth"&&<div className="modal-backdrop"><form className="auth-modal" onSubmit=
{signIn}><button type="button" className="modal-close" onClick={()=>setPage("home")}><X/>
</button><div className="brand">UNBLINDED<span>.</span></div><span
className="eyebrow">STUDENT ACCOUNT</span><h2>{authMode==="login"?"Welcome back":"Create
your account"}</h2><p>One space for your learning journey.</p>{authMode==="signup"&&<input
required placeholder="Your name" value={auth.name} onChange=
{e=>setAuth({...auth,name:e.target.value})}/>}<input required type="email"
placeholder="Email address" value={auth.email} onChange=
{e=>setAuth({...auth,email:e.target.value})}/><input required minLength="8" type="password"
placeholder="Password (8+ characters)" value={auth.password} onChange=
{e=>setAuth({...auth,password:e.target.value})}/><button className="primary-button full-
button" disabled={busy}>{busy?"Please wait...":authMode==="login"?"Sign in":"Create
account"}</button><button type="button" className="text-action centered" onClick=
{()=>setAuthMode(authMode==="login"?"signup":"login")}>{authMode==="login"?"New to
UNBLINDED? Create an account":"Already registered? Sign in"}</button></form></div>}
  </div>;
}

function StudentProfile({ session, profile, updateStudent }) {
  const
[details,setDetails]=useState({education_level:"",institution:"",country:"Bangladesh",intere
sts:""});
  useEffect(()=>
{if(!session)return;supabase.from("student_profiles").select("*").eq("user_id",session.user.
id).maybeSingle().then(({data})=>
{if(data)setDetails({education_level:data.education_level||"",institution:data.institution||
"",country:data.country||"Bangladesh",interests:data.interests||""});});},[session]);
  const save=async e=>{e.preventDefault();for(const [k,v] of Object.entries(details))await
updateStudent(k,v);};
  return <section className="surface-card profile-editor"><div className="profile-heading">
<div className="avatar big">{(profile?.display_name||"S").slice(0,1)}</div><div><h2>
{profile?.display_name||"Student"}</h2><p>{session.user.email}</p><small>Only share details
you are comfortable making available.</small></div></div><form onSubmit={save}>
<label>Education level<select value={details.education_level} onChange=
{e=>setDetails({...details,education_level:e.target.value})}><option value="">Choose your
level</option>{["School","College","University","Professional"].map(v=><option key={v}>{v}
```

```
</option>)}</select></label><label>Institution (optional)<input value={details.institution}
onChange={e=>setDetails({...details,institution:e.target.value})} placeholder="Your school,
college or university"/></label><label>Country<select value={details.country} onChange=
{e=>setDetails({...details,country:e.target.value})}><option>Bangladesh</option>
<option>Other</option></select></label><label>Interests and subjects<input value=
{details.interests} onChange={e=>setDetails({...details,interests:e.target.value})}
placeholder="e.g. Physics, design, economics"/></label><button className="primary-
button">Save profile</button></form></section>;
}

function Root() {
  const [authVisible,setAuthVisible]=useState(false);
  const [session,setSession]=useState(null);
  useEffect(()=>
{if(!configured)return;supabase.auth.getSession().then(({data})=>setSession(data.session));c
onst {data:
{subscription}}=supabase.auth.onAuthStateChange((_e,s)=>setSession(s));return()=>subscriptio
n.unsubscribe();},[]);
  if(!configured)return <App/>;
  if(!session)return <Landing onSignIn={()=>setAuthVisible(true)} authVisible={authVisible}
setAuthVisible={setAuthVisible}/>;
  return <App/>;
}
function Landing({onSignIn,authVisible,setAuthVisible}) {
  const [signup,setSignup]=useState(false),
[form,setForm]=useState({name:"",email:"",password:""}),[busy,setBusy]=useState(false),
[message,setMessage]=useState("");
  async function submit(e){e.preventDefault();setBusy(true);let r;if(signup){r=await
supabase.auth.signUp({email:form.email,password:form.password,options:{data:
{display_name:form.name}}});if(!r.error&&r.data.user)await
supabase.from("profiles").upsert({id:r.data.user.id,display_name:form.name});}else r=await
supabase.auth.signInWithPassword({email:form.email,password:form.password});setBusy(false);s
etMessage(r.error?.message||(signup?"Account created. Check your email if confirmation is
enabled.":"Welcome back."));}
  return <div className="landing"><nav className="landing-nav"><a
className="brand">UNBLINDED<span>.</span></a><div><a href="#platform">Platform</a><a
href="#why">Why UNBLINDED</a><button className="secondary-button" onClick={()=>
{setSignup(false);setAuthVisible(true)}}>Sign in</button><button className="primary-button"
onClick={()=>{setSignup(true);setAuthVisible(true)}}>Get started <ArrowRight size={15}/>
</button></div></nav><main className="landing-hero"><div className="landing-copy"><span
className="eyebrow">A STUDENT PLATFORM · BANGLADESH FIRST</span><h1>Your space to<br/>
<em>learn, connect</em><br/>and grow.</h1><p>One thoughtful workspace for learning
resources, student community, academic planning and opportunities.</p><div
className="landing-actions"><button className="primary-button" onClick={()=>
{setSignup(true);setAuthVisible(true)}}>Create your free account <ArrowRight size={16}/>
</button><a href="#platform" className="text-action">Explore the platform</a></div><div
className="landing-proof"><span><Check size={14}/> Built for students</span><span><Check
size={14}/> Bangladesh first</span><span><Check size={14}/> Made to grow with you</span>
</div></div><div className="landing-preview"><div className="preview-top"><span>UNBLINDED /
OVERVIEW</span><span className="preview-status">● YOUR WORKSPACE</span></div><div
className="preview-greeting"><small>MONDAY, YOUR NEXT CHAPTER</small><h2>Make today<br/>
<em>count.</em></h2></div><div className="preview-stats"><div><small>STUDY GOALS</small>
<b>04</b></div><div><small>RESOURCES</small><b>128</b></div><div>
<small>OPPORTUNITIES</small><b>12</b></div></div><div className="preview-lines"><span/>
<span/><span/></div></div></main><section id="platform" className="landing-features"><span
className="eyebrow">THE STUDENT WORKSPACE</span><h2>Everything you need<br/>to move forward.
</h2><div className="feature-grid">{[[BookOpen,"Learn","Explore notes, guides and academic
resources."],[Users,"Community","Ask questions and learn alongside other students."],
[CalendarDays,"Planner","Organize assignments, deadlines and study goals."],[Zap,"Study
tools","Build focus habits with simple academic utilities."],
[Trophy,"Opportunities","Discover scholarships, competitions and more."],
[MessageCircle,"Connect","Keep meaningful student conversations
together."]].map(([Icon,title,desc])=><div className="feature-tile" key={title}><Icon/><h3>
{title}</h3><p>{desc}</p></div>)}</div></section><footer className="landing-footer">
<b>UNBLINDED<span>.</span></b><span>Your space to learn, connect and grow.</span><span>©
```

```
2026 UNBLINDED</span></footer>{authVisible&&<div className="modal-backdrop"><form
className="auth-modal" onSubmit={submit}><button type="button" className="modal-close"
onClick={()=>setAuthVisible(false)}><X/></button><div className="brand">UNBLINDED<span>.
</span></div><span className="eyebrow">STUDENT ACCOUNT</span><h2>{signup?"Create your
account":"Welcome back"}</h2>{signup&&<input required placeholder="Your name" value=
{form.name} onChange={e=>setForm({...form,name:e.target.value})}/>}<input required
type="email" placeholder="Email address" value={form.email} onChange=
{e=>setForm({...form,email:e.target.value})}/><input required minLength="8" type="password"
placeholder="Password (8+ characters)" value={form.password} onChange=
{e=>setForm({...form,password:e.target.value})}/><button className="primary-button full-
button" disabled={busy}>{busy?"Please wait...":signup?"Create account":"Sign in"}</button>
{message&&<p className="form-message">{message}</p>}<button type="button" className="text-
action centered" onClick={()=>setSignup(!signup)}>{signup?"Already registered? Sign in":"New
here? Create an account"}</button></form></div>}</div>;
}
createRoot(document.getElementById("root")).render(<Root/>);

// Nothing to see .
```